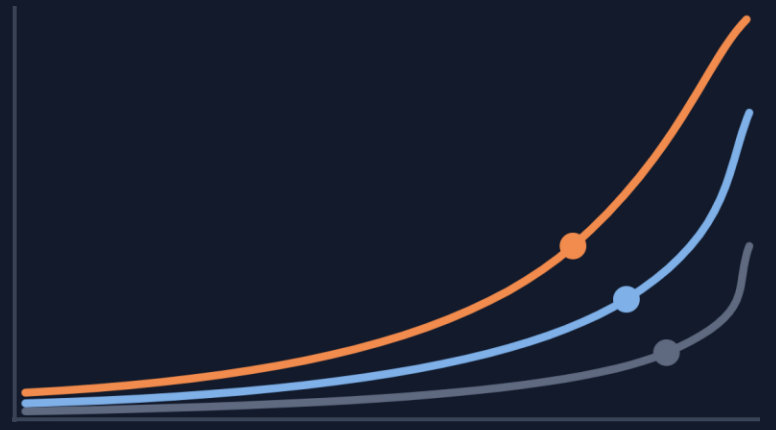
LLM Serving Engine Benchmark

Architecture, implementation plan and tool stack for choosing the right inference engine — backed by reproducible measurements.

vLLM SGLang TensorRT-LLM AIPerf AISimulate



THE GOAL

Pick the engine with evidence, not opinion

A reproducible Python tool that deploys **the same model** on different serving engines, replays realistic traffic, and recommends an engine and configuration for each workload, GPU and latency target.

What we measure

- Speed, capacity and GPU resource use
- General, coding and agentic traffic

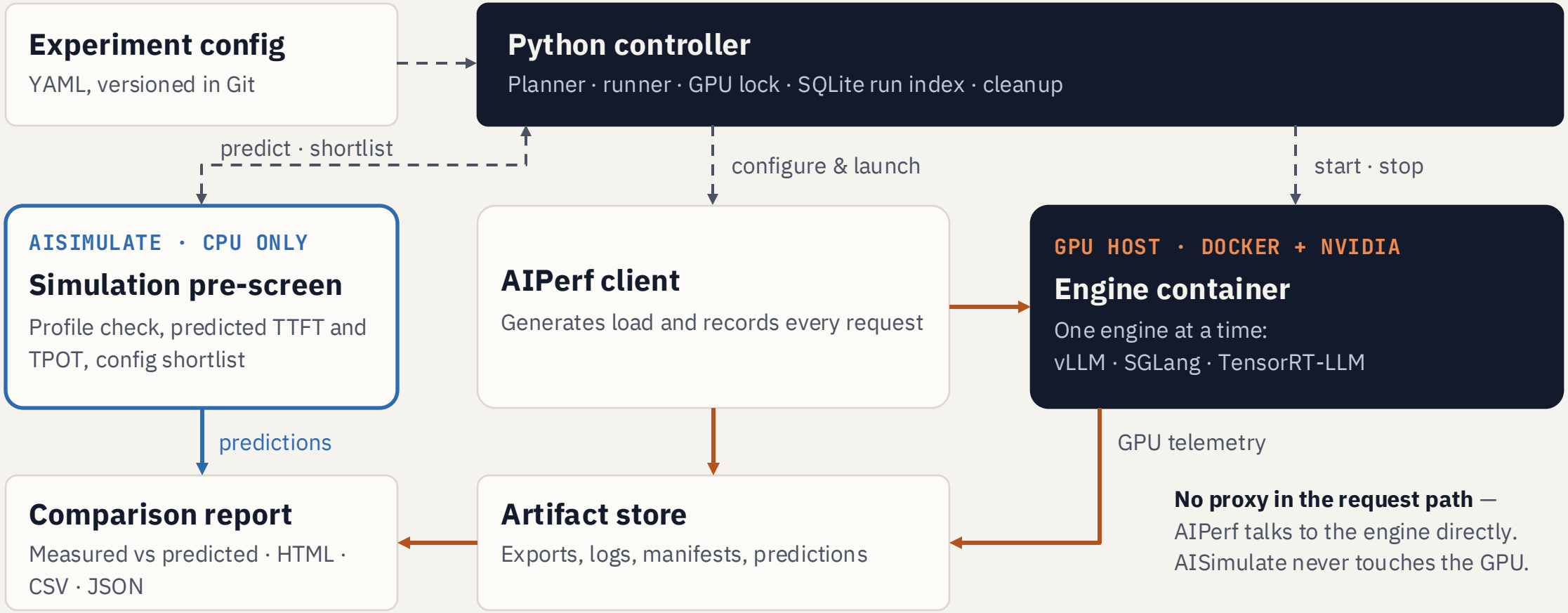
Out of scope

- Grading the quality of answers
- Executing generated code for correctness
- Judging whether an agent solved a task

SYSTEM ARCHITECTURE

Control · Traffic & evidence · Simulated predictions

Simulate to shortlist, measure to decide



SIMULATION PRE-SCREEN

Check the model profile before simulating

STEP 1 · SUPPORT CHECK

Does AISimulate have a profile for our setup?

Model · GPU system · engine backend · backend version

YES

PROFILE FOUND

Predict and shortlist

Estimate TTFT, TPOT and throughput per config, then send only the strongest few to real GPU runs.

NO

NOT SUPPORTED

Skip simulation

Run the full measured sweep. The benchmark works the same, it just uses more GPU hours.

Real GPU runs with AIPerf make the final decision

HOW TO CHECK

```
$ pip install aisimulate
$ aiconfigurator cli support \
    --model-path <MODEL_ID> \
    --system <GPU_SYSTEM> \
    --backend <vllm|sglang|trtllm>
```

- Run it for every engine you plan to compare.
- Read the YES/NO result: exit code 0 only means the query ran.
- Pin the AISimulate version: this command is due to change in 0.14.0.

Fairness rule: use the shortlist only when every compared engine has a profile.

COMPONENTS

Nine packages, each with one job

`config/`
Config & planner
Validate YAML, pin revisions, expand cells, estimate budget.

`workloads/`
Workloads
Import datasets and traces, keep sessions, emit hashed manifests.

`simulation/`
AISimulate bridge
Check profiles, run predictions, save the shortlist.

`engines/`
Engine adapters
Check capabilities, render engine-specific Docker arguments.

`runner/`
Runner
GPU lock, lifecycle, deadlines, cancel, resume and cleanup.

`benchmarks/`
AIPerf bridge
Translate our schema to a pinned AIPerf config.

`metrics/`
Metrics
Collect AIPerf exports and GPU telemetry; normalize units.

`analysis/`
Analysis
Validate, aggregate repeats, score SLOs, measure prediction error.

`reports/`
Reports
Self-contained HTML with Jinja2 and Plotly, plus CSV and JSON.

All under src/engine_bench/ — plus **configs**, **manifests**, **tests**, **docs** and CI workflows.

WORKLOADS

Three profiles, replayed identically

General

`WildChat`

Real user conversations — short and long prompts, single and multi-turn.

Sampled by language, input size and turn count; full sessions kept.

Coding

`InstructCoder`

Code-editing instructions with larger inputs and longer responses.

Sent as inference inputs only — no code execution or quality score.

Agentic

`TraceLab replay`

Call sequences with growing context, repeated prefixes and tool delays.

Sessions kept intact; AIPerf synthesizes content to recorded token counts.

LOAD AND METRICS

Load patterns and what we measure

Closed-loop concurrency

1, 5, 10 and 20 in-flight requests; a new one starts as each finishes.

Open-loop request rate

Arrivals independent of completions; finds sustainable req/s and records backlog.

Session replay

Dependent turns with recorded think and tool delays, on their own schedule.

Metric	What it tells us
P50 / P95 TTFT	Dispatch to first token, incl. queueing
P95 end-to-end	Dispatch to completed response
Generation · TPOT	Decode speed per request
Output throughput	Generated tokens per second over the run
Goodput	Requests meeting every limit, per second
Errors	HTTP failures, timeouts, OOMs, broken streams
GPU resources	VRAM, utilization, clocks and power

Starting SLO · proposal

P95 TTFT ≤ 1,500 ms · errors ≤ 1% · 60 s deadline

DECISION LOGIC

From measurements to a recommendation

1. **Reject incomparable cells**
   Different model or precision, unverified hardware, silent truncation, incomplete records or a saturated client.

2. **Filter by the objectives**
   Group matching modes only; apply latency limits, joint attainment and the error budget.

3. **Compare at the required load**
   Latency and cost on passing deployments; never extrapolate past the tested boundary.

4. **Aggregate repetitions**
   At least three trials, each reported, with confidence intervals from session-aware resampling.

THREE VALID OUTCOMES

Winner
Stable benefit above the predeclared 5% practical threshold.

Inconclusive
Returns the next load points or repetitions needed.

No feasible configuration
Nothing passes the latency and error limits.

AISimulate predictions are never evidence. They only decide what we test first; the recommendation comes from measured cells, and the report shows predicted vs measured error.

TOOL STACK

The tools we will use

ORCHESTRATION
Python
Controller and the engine-bench CLI

MEASUREMENT
AIPerf
Load generator and client, pinned release

ENGINES · MVP
vLLM + SGLang
The two engines compared first

SIMULATION
AISimulate
Profile check and predictions, CPU only

ENGINE · NEXT
TensorRT-LLM
Added after a compatibility check

RUNTIME
Docker + NVIDIA
One engine container at a time on the GPU

STATE
SQLite
Run index, state transitions, resume

CONFIG
YAML + GitHub
Versioned experiment definitions

REPORTING
Jinja2 + Plotly
Self-contained HTML reports with charts

CI
GitHub Actions
CPU checks on PRs, manual GPU runs

TELEMETRY
nvidia-smi · DCGM
GPU memory, utilization, clocks, power

DATA
Workload data
WildChat, InstructCoder, TraceLab replay

REPOSITORY AND CI

One CLI, one repo, controlled GPU runs

COMMAND SURFACE

```
$ engine-bench validate configs/experiment.yaml
$ engine-bench simulate configs/experiment.yaml
$ engine-bench plan configs/experiment.yaml
$ engine-bench run configs/experiment.yaml
$ engine-bench resume EXPERIMENT_ID
$ engine-bench report EXPERIMENT_ID
```

EXPERIMENT.YAML · EXCERPT

```
engines: [vllm_baseline, sglang_baseline]
concurrency: [1, 5, 10, 20]
repetitions: 3
ttft_p95_ms: 1500
```

Pull requests · CPU

Validate schemas, lint code, test export fixtures and produce a dry-run plan.

GPU experiments · manual

Trusted workflow on a self-hosted runner; pinned actions and images; a concurrency group per GPU plus the local lock.

Published artifacts

HTML, CSV, decision JSON and redacted manifests. Weights, secrets and customer data stay out of Git.